Unit 04: Inheritance

CONTENTS

Objectives

After studying this unit, you will be able to:

- Recognize the inheritance
- Describe the different types of inheritance
- Analyze and Explain making private member inheritable

Introduction

The ability to reuse code is a key element of OOP. The concept of reusability is highly supported in C++. The C++ classes can be reused in a variety of ways. Other programmers can use a class once it has been written and tested. The properties Notes of existing classes can be reused to create new classes.

INHERITANCE is the process of creating a new class from an existing one. Because every object of the defined class "is" also an object of the inherited class type, this is sometimes referred to as a "IS-A" relationship. The previous class is known as the 'BASE' class, whereas the new class is known as the 'DERIVED' class or sub-class.

Notes:

- The capability of a class to derive properties and characteristics from another class is called Inheritance.
- Inheritance is a process in which one object acquires all the properties and behaviours of its parent object automatically

4.1 Defining Derived Class

A derived class is specified by defining its relationship with the base class in addition to its owndetails. The general syntax of defining a derived class is as follows:

Class derived class name : access_specifier base class name

{

……

…… // members of derivedclass

}

The colon (:) indicates that the derivedclassname class is derived from the baseclassname class. The access_specifier or the visibility mode is optional and, if present, may be public, private or protected. By default it is private. Visibility mode describes the accessibility status of derived features. For example,

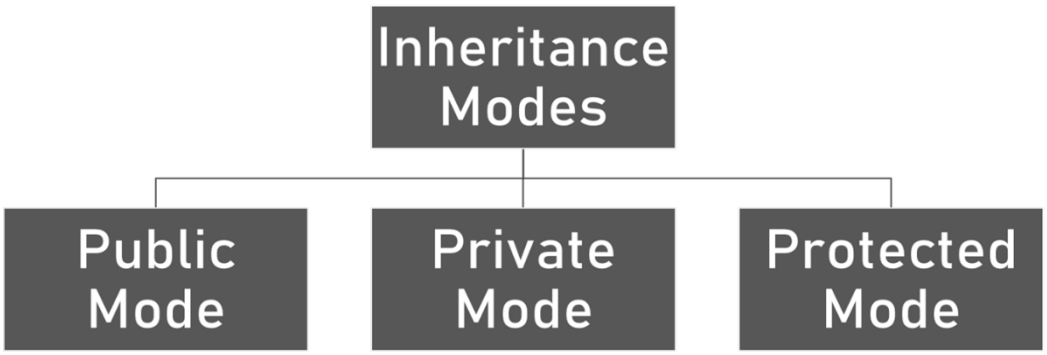
```
class xyz //base class
{
//members of xyz
};
class ABC: public xyz //public derivation
{
//members of ABC
};
class ABC : XYZ //private derivation (by default)
{
//members of ABC
};
```

In inheritance, some of the base class data elements and member functions are inherited into the derived class and some are not. We can add our own data and member functions and thus extend the functionality of the base class.

4.2 Modes of Inheritance

Inheritance Modes:-

1. Public Mode
2. Private Mode
3. Protected Mode



1. **Public Mode:** If we derive a sub class from a public base class. Then the public member of the base class will become public in the derived class and protected members of the base class will become protected in derived class.
2. **Private Mode:** If we derive a sub class from a Private base class. Then both public member and protected members of the base class will become Private in derived class.
3. **Protected Mode:** If we derive a sub class from a protected base class. Then both public member and protected members of the base class will become protected in derived class.

Did you know?

What are the advantages of inheritance?

Inheritance offers the following advantages:

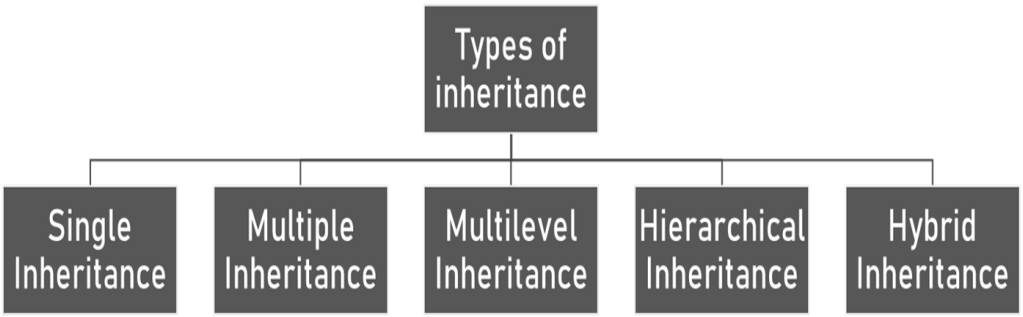
Development model closer to real life object model with hierarchical relationships

Reusability — facility to use public methods of base class without rewriting the same

Extensibility — extending the base class logic as per business logic of the derived class

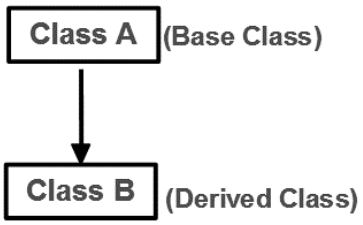
Data hiding — base class can decide to keep some data private so that it cannot be alteredby the derived class.

4.3 Types of Inheritance in C++



1. Single Inheritance

When a class inherits from a single base class, it is referred to as single inheritance.



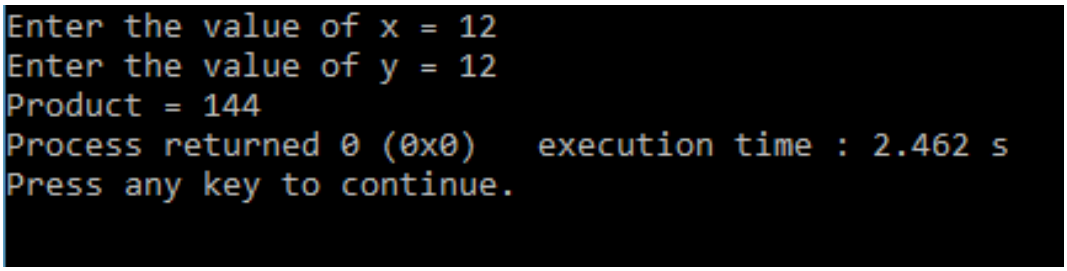
Following program shows working of single inheritance.

Example

```
#include <iostream>
using namespace std;
class base
```

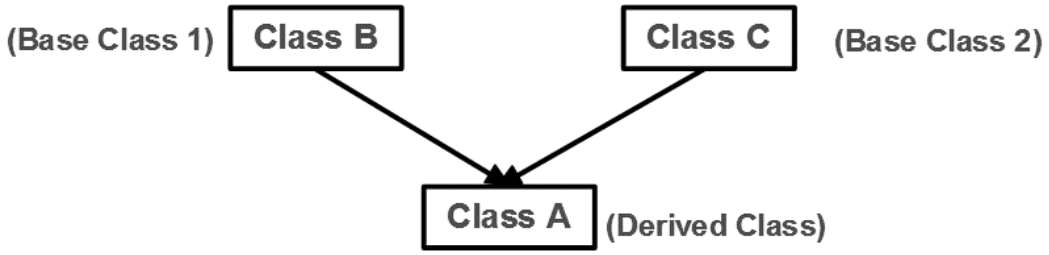
```
{
public:
int x;
voidgetdata()
  {
cout<< "Enter the value of x = "; cin>> x;
  }
 };
class derive : public base
{
private:
int y;
public:
voidreaddata()
  {
cout<< "Enter the value of y = "; cin>> y;
  }
void product()
  {
cout<< "Product = " << x * y;
  }
 };

int main()
 {
derive a;
a.getdata();
a.readdata();
a.product();
return 0;
 }
```

Output



2. Multiple Inheritance

When a class inherits from a two base classes, it is referred to as multiple inheritance.



Following program shows working of multiple inheritance.

Example

```
#include<iostream>
using namespace std;
class A
{
        public:
        int x;
        voidget_data()
  {
cout<< "enter value of x: ";
cin>> x;
  } };
class B
{       public:
        int y;
        void get_data1()
        {
        cout<< "enter value of y: "; cin>> y;
        }   };
class C : public A, public B
{
        public:
        void sum()
        {
        cout<< "Sum = " << x + y;
        }
};
```

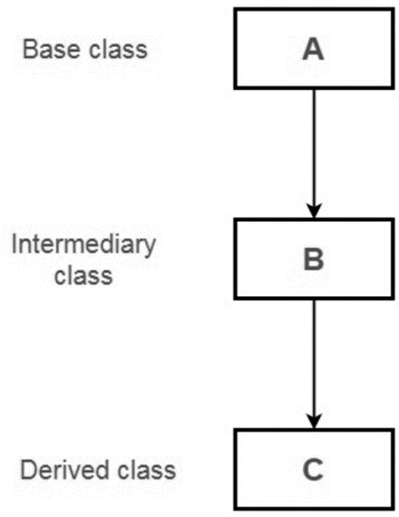
```
int main()
{
        C obj;
      obj.get_data();
       obj.get_data1();
      obj.sum();
      return 0;
}       //end of program
```

Output

```
enter value of x: 25
enter value of y: 35
Sum = 60
Process returned 0 (0x0)   execution time : 2.871 s
Press any key to continue.
```

3. Multilevel Inheritance

If a class is derived from another derived class then it is called multilevel inheritance.



The declaration for the same would be:

```
Class A
{
//body
}
Class B : public A
{
//body
```

```
}
Class C : public B
{
//body
}
```

Following program shows working of multilevel inheritance.

 Example

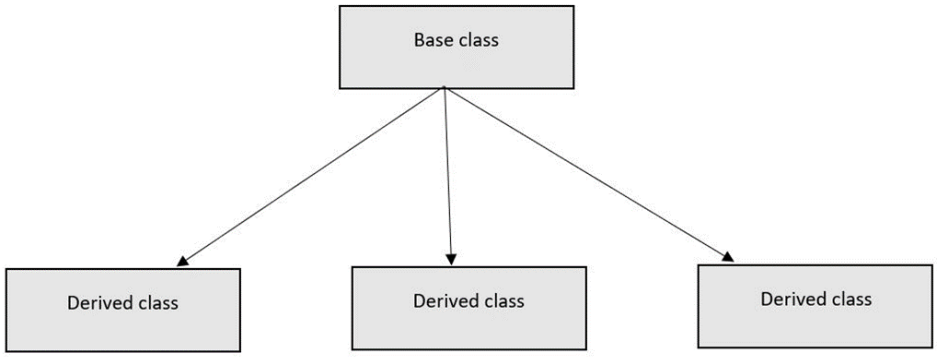
```
#include<iostream>
using namespace std;
class A{
public:
int marks;
voidget_data(){
cout<<"Enter Marks";
cin>>marks;
   }
   };
class B:public A
   {
public:
intshow_data(){
cout<<"Entered Marks: " <<marks;
      }
   };

class C:public B{
   };
main(){
   C obj;
obj.get_data();
obj.show_data();
   }
```

Output

```
Enter Marks50
Entered Marks: 50
Process returned 0 (0x0)   execution time : 2.506 s
Press any key to continue.
```

4. Hierarchical Inheritance

Hierarchical inheritance is a kind of inheritance where more than one class is inherited from a single parent or base class. Especially those features which are common in the parent class is also common with the base class.



Following program shows working of hierarchical inheritance.

Example

```
#include<iostream>
using namespace std;
class A
{
        public:
        intx,y;
        voidget_data()
  {
        cout<< "Enter value of x: ";
        cin>> x;
        cout<<"Enter value of y: ";
        cin>>y;
  }
};
class B:public A
{
```

```
        public:

        voidshow_sum()
        {
        cout<< "Sum of x and y is : "<<x+y<<endl;
        }
};
class C : public A
{
        public:
        voidshow_product()
        {
        cout<< "Product of x and y is : "<<x*y;
        }
};

int main()
{   B obj1;
          C obj2;
         obj1.get_data();
   obj1.show_sum();
         obj2.get_data();
         obj2.show_product();

        return 0;
}
```

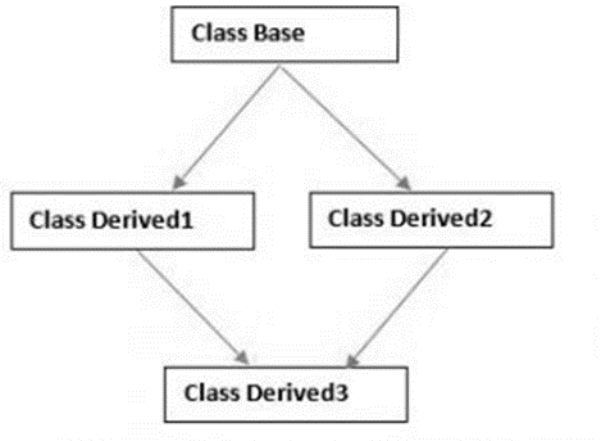
Output

```
Enter value of x: 15
Enter value of y: 2
Sum of x and y is : 17
Enter value of x: 35
Enter value of y: 2
Product of x and y is : 70
Process returned 0 (0x0)   execution time : 5.966 s
Press any key to continue.
```

5. Hybrid Inheritance

There could be situations where we need to apply two or more types of inheritance to design a program. Basically Hybrid Inheritance is the combination of one or more types of the inheritance.

Here is one implementation of hybrid inheritance. Hybrid inheritance is combination of two or more types of inheritance. It is also known as multipath inheritance.



Following program shows working of hybrid inheritance.

Example

```
#include <iostream>
using namespace std;
class A
{
        public:
        int x;
};
class B : public A
{
        public:
        B()
        {
          x = 500;
        }
};
class C
 {
        public:
        int y;
        C()
        {
           y = 100;
     }
};
```

```
class D : public B, public C
{
        public:
        void sum()
        {
        cout<< "Sum= " << x + y;
        }
};

int main()
{
   D obj1;
        obj1.sum();
        return 0;
}
```

Output

```
Sum= 600
Process returned 0 (0x0)   execution time : 0.087 s
Press any key to continue.
```

4.4 Making a Private Member Inheritable

The members of base class which are inherited by the derived class and their accessibility is determined by visibility modes. Visibility modes are:

1. Private: When a base class is privately inherited by a derived class, 'public members' of the base class become private members of the derived class and therefore the publicmembers of the base class can be accessed by its own objects using the dot operator. Theresult is that we have no member of base class that is accessible to the objects of thederived class.

2. Public: On the other hand, when the base class is publicly inherited, 'public members' ofthe base class become 'public members' of derived class and therefore they are accessibleto the objects of the derived class.

3. Protected: C++ provides a third visibility modifier, protected, which serve a little purposein the inheritance. A member declared as protected is accessible by the member functionswithin its class and any class immediately derived from it. It cannot be accessed by functionsoutside these two classes.

Private derivation means that the base class has been inherited privately. Public members and protected members of the base class are private within the derived class.Private members of the base class stay private within the base class.

Syntax

```
classbase_class_name
{
-----
-----
}
```

```
classderived_class_name : private base_class_name
{
----
----
}

//          Program

#include<iostream>
using namespace std;
classemp{
private:
int id;
char name[10];
int salary;
voidget_data(){
cout<<"Enter Id,Name and Salary of  employee"<<endl;
cin>>id>>name>>salary;

   }
public:
voiddisp(){
get_data();
cout<<"Details are"<<endl;
cout<<"Emp ID   "<< id<<endl<<"Emp Name  " <<name  <<endl<<"Emp Salary "<<salary;

   }
};
main(){
empobj;
obj.disp();

}
```

Output

```
Enter Id,Name and Salary of  employee
12
John
15000
Details are
Emp ID   12
Emp Name  John
Emp Salary 15000
Process returned 0 (0x0)   execution time : 7.479 s
Press any key to continue.
```

Summary

Inheritance is the capability of one class to inherit properties from another class.

It supports reusability of code and is able to simulate the transitive nature of real lifeobjects. Inheritance has many forms: Single inheritance, multiple inheritance, hierarchicalinheritance, multilevel inheritance and hybrid inheritance.

A subclass can derive itself publicly, privately or protected. The derived class constructoris responsible for invoking the base class constructor, the derived class can directly accessonly the public and protected members of the base class.

When a class inherits from more than one base class, this is called multiple inheritance.

A class may contain objects of another class inside it. This situation is called nesting of objects and in such a situation, the contained objects areconstructed first before constructing the objects of the enclosing class.

Single Inheritance: Where a class inherits from a single base class, it is known as singleinheritance.

Multilevel Inheritance: When the inheritance is such that the class. A serves as a base classfor a derived class B which is turn serves as a base class for the derived class C. This typeof inheritance is called 'Multilevel Inheritance.

Multiple Inheritance: A class inherit the attributes of two or more classes. This mechanism is known as Multiple Inheritance.'

Hybrid Inheritance: The combination of one or more types of the inheritance.

Keywords

Abstract Class: A class serving only a base class for other classes and no objects of which are created.

Base class: A class from which another class inherits. (Also called super class) Containership: The relationship of two classes such that the objects of a class are enclosed within the other class.

Derived class: A class inheriting properties from another class. (also called sub class).

Inheritance: Capability of one class to inherit properties from another class.

Inheritance Graph: The chain depicting relationship between a base class and derived class.

Visibility Mode: The public, private or protected specifier that controls the visibility and availability of a member in a class.

Self Assessment

1. Inheritance allow in C++ program?

A. Class Re-usability
B. Creating a hierarchy of classes
C. Extendibility
D. All of above

2. Functions that can be inherited from base class in C++ program.

A. Constructor
B. Destructor
C. Static function
D. None of above

3. A class can inherit properties of another class which is known as …….. Inheritance.

A. Single
B. Multiple
C. Multilevel
D. Hierarchical

4. What is the syntax of inheritance of class?
A. class name
B. class name: access specifer
C. class name : access specifer class name
D. None of above

5. Members which are not intended to be inherited are declared as_____
A. Public members
B. Protected members
C. Private Members
D. Private or protected members

6. While inheriting a class, if no access mode is specified, then which among the following is true in C++?
A. It gets inherited publicly by default
B. It gets inherited protected by default
C. It gets inherited privately by default
D. It is not possible.

7. How can you make the private members inheritable?
A. By making their visibility mode as public only
B. By making their visibility mode as protected only
C. By making their visibility mode as private in derived class
D. Can be done both by making the visibility mode public or protected

8. What is meant by multiple inheritance?
A. Deriving a base class from derived class
B. Deriving a derived class from base class
C. Deriving a deriving class from more than one base class
D. None of above

9. Which symbol is used to create multiple inheritance?
A. Dot
B. Comma
C. Dollar
D. None of the above

10. Which among the following best define multilevel inheritance?
A. A class derived from another derived class
B. Classes being derived from another derived class
C. Continuing single level inheritance
D. Class which have more than one parent

11. All the classes must have all the members declared private to implement multilevel inheritance.
A. True
B. False
C. Sometimes true, sometimes false
D. Always false

12. Which among the following is best to defined hierarchical inheritance?
A. More than one classes being derived from one class
B. More than two classes being derived from single base class
C. At most two classes being derived from single base class
D. At most 1 class derived from another class

13. How many classes must be there to implement hierarchical inheritance?
A. Exactly 3
B. At least 3
C. At most 3
D. At least 1

14. Which type of inheritance must be used so that the resultant is hybrid?
A. Multiple
B. Hierarchical
C. Multilevel
D. None of the above

15. What is the minimum number of classes to be there in a program implemented hybrid inheritance?
A. 2
B. 3
C. 4
D. No limit

Review Questions

1. What do you mean by inheritance? Explain different types of inheritance with suitable example.
2. Consider a situation where three kinds of inheritance are involved. Explain this situation with an example.
3. What is the difference between protected and private members?
4. Scrutinize the major use of multilevel inheritance.
5. Discuss a situation in which the private derivation will be more appropriate as compared to public derivation.
6. Write a C++ program to read and display information about employees and managers. Employee is a class that contains employee number, name, address and department. Manager class and a list of employees working under a manager.
7. Differentiate between public and private inheritances with suitable examples.
8. Explain how a sub-class may inherit from multiple classes.
9. What is the purpose of virtual base classes?
10. Write a C++ program that demonstrate working of hybrid inheritance.

Answers for Self Assessment

1.	D	2.	D	3.	A	4.	C	5.	C
6.	C	7.	D	8.	C	9.	B	10.	B
11.	B	12.	A	13.	B	14.	D	15.	D

Further Readings

E Balagurusamy; Object-oriented Programming with C++; Tata McGraw-Hill.

Herbert Schildt; The Complete Reference C++; Tata McGraw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.

web Links

http://msdn.microsoft.com/en-us/library/wcz57btd(v=vs.80).aspx

http://www.learncpp.com/cpp-tutorial/117-multiple-inheritance/